

BITS PILANI
Work Integrated Learning Programme
M.Tech – Software Engineering

Introduction to DevOps (SEZG514)

Assignment 1

ACEest Fitness & Gym Management System

DevOps CI/CD Implementation Project

Submitted by:

Sarath Kumar S (2024TM93636)

Semester: 3

Date: 2 April 2026

Project Repository

GitHub: <https://github.com/2024TM93636/aceest-fitness-ci-cd>

Index

Index.....	2
1. Project Overview	3
2. Tech Stack	3
3. Application Development.....	3
4. Project Architecture.....	4
5. Project Structure	4
6. Application Features.....	5
REST API Endpoints.....	5
UI Features.....	5
7. Version Control	6
8. Unit Testing.....	6
9. Docker Containerization	7
10. Jenkins Pipeline	8
11. GitHub Actions CI Pipeline	10
12. CI/CD Workflow Diagram	11
13. DevOps Practices Implemented	11
14. Future Enhancements.....	11
15. Conclusion	12
16. References.....	12

1. Project Overview

This project demonstrates the implementation of a **complete DevOps CI/CD pipeline** for a Flask-based Fitness & Gym Management API. The project focuses on ensuring **code quality, consistency, and automated delivery** through version control, automated testing, containerization, and CI/CD pipelines.

Objectives:

- Build a modular Flask application
- Implement Git-based version control
- Develop and run Pytest unit tests
- Containerize the application using Docker
- Implement CI/CD pipelines with GitHub Actions and Jenkins
- Automate build, test, and deployment processes ensuring reliability and consistency

2. Tech Stack

Category	Technology
Backend	Python, Flask
Testing	Pytest
Containerization	Docker
CI/CD	GitHub Actions, Jenkins
Version Control	Git, GitHub

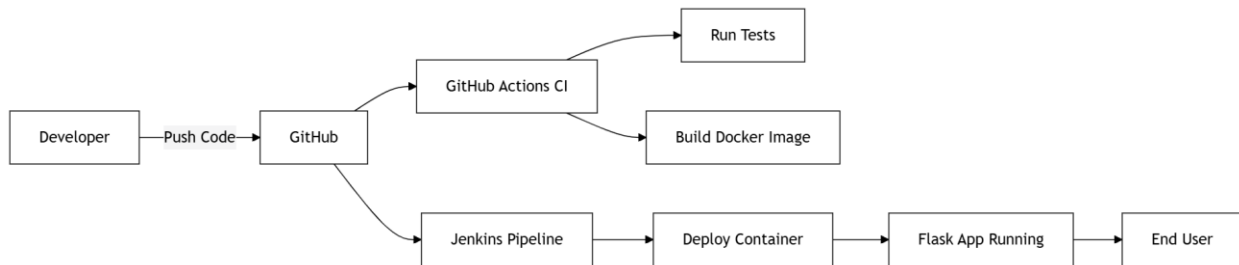
3. Application Development

- The application is developed using Flask following a modular structure.
- It provides RESTful APIs for managing gym members and workouts.
- The application supports CRUD operations and follows a clean separation of concerns.

4. Project Architecture

The overall architecture ensures smooth DevOps workflow from development to production:

Flow: Developer → GitHub → GitHub Actions CI → Jenkins Build → Docker Container → Flask Application → End User



5. Project Structure

```
aceest-fitness-ci-cd/
├── app.py                # Flask API implementation
├── requirements.txt       # Python dependencies
├── Dockerfile            # Container configuration
├── Jenkinsfile           # Jenkins CI/CD pipeline
├── README.md             # Project documentation
├── tests/
│   └── test_app.py       # Pytest unit tests
├── .github/
│   └── workflows/
│       └── ci.yml        # GitHub Actions CI pipeline
```

Explanation:

- `app.py` – Core Flask application with member and workout APIs
- `requirements.txt` – Python package dependencies

- Dockerfile – Container setup
- Jenkinsfile – Build and deploy pipeline
- tests/test_app.py – Unit tests using Pytest
- .github/workflows/ci.yml – GitHub Actions CI pipeline

6. Application Features

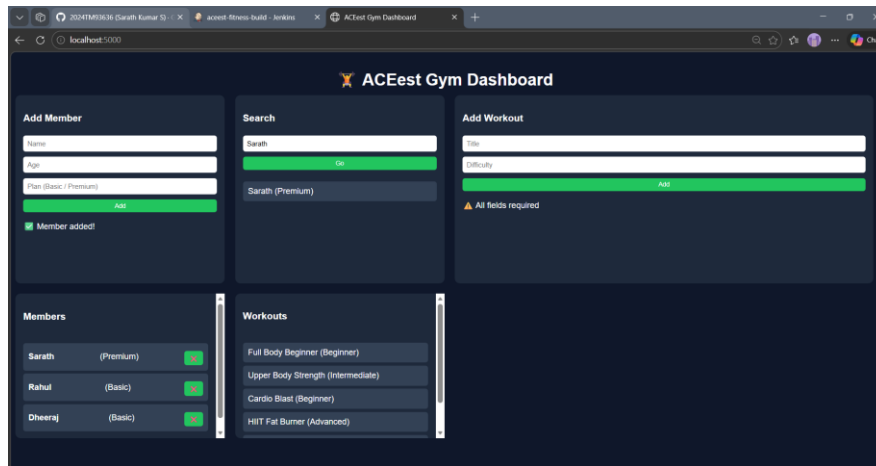
REST API Endpoints

Endpoint	Method	Description
/	GET	Home Dashboard
/api/members	GET	Retrieve all members
/api/members/<id>	GET	Retrieve member by ID
/api/members	POST	Add a new member
/api/members/<id>	PUT	Update member details
/api/members/<id>	DELETE	Delete a member
/api/workouts	GET	List all workouts
/api/workouts	POST	Create a new workout
/api/members/<member_id>/assign/<workout_id>	POST	Assign workout to a member
/api/members/search?name=	GET	Search members by name

UI Features

- Dashboard to view members and workouts
- Add, update, and delete members
- Add workouts and assign workouts to members
- Search functionality for members

UI Screenshot:



7. Version Control

GitHub Repository: <https://github.com/2024TM93636/aceest-fitness-ci-cd>

Branching Strategy:

- main – Production-ready code
- feature/* – New features and bug fixes

Sample Commits:

- Added member and workout API endpoints
- Implemented Pytest unit tests for member and workout APIs
- Dockerfile and Jenkinsfile added for CI/CD
- Configured GitHub Actions CI pipeline

8. Unit Testing

- Integrated **Pytest** for unit testing
- Tests cover members CRUD, workouts CRUD, assignment, and search functionality

Sample Test Command: `pytest`

Test Results Screenshot:

```
1 Run Tests
11 ===== test session starts =====
12 platform linux -- Python 3.10.20, pytest-9.0.2, pluggy-1.6.0
13 rootdir: /home/runner/work/aceest-fitness-ci-cd/aceest-fitness-ci-cd
14 collected 10 items
15
16 tests/test_app.py ..... [100%]
17
18 ===== 10 passed in 0.16s =====
```

Tests are automatically executed in the CI pipeline to prevent faulty code from reaching the build stage.

9. Docker Containerization

Dockerfile:

- Base image: python:3.10-slim
- Install dependencies from requirements.txt
- Expose port 5000 and run app.py

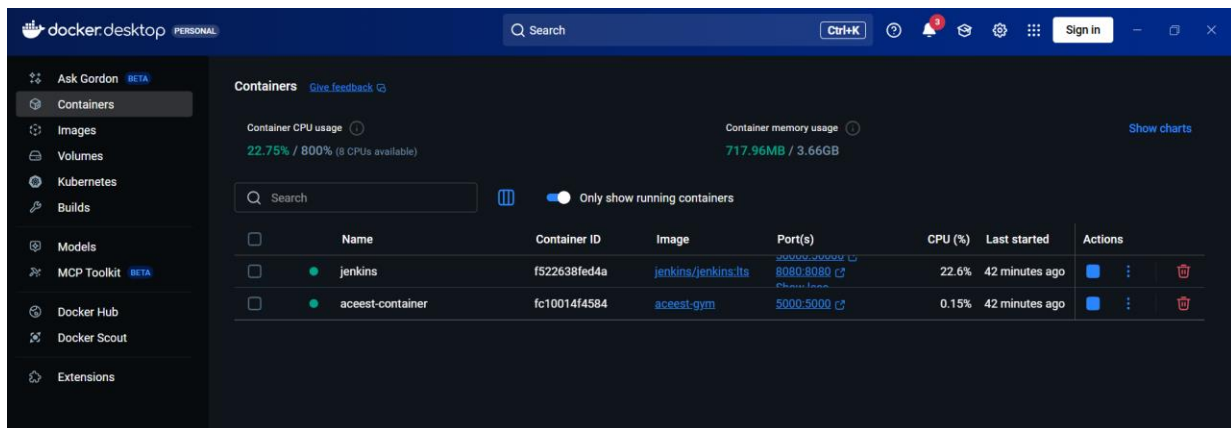
Commands:

- docker build -t aceest-gym .
- docker run -p 5000:5000 aceest-gym

```
(venv) PS D:\M.Tech\BITS Pilani\Semester 3\Introduction to Devops\Assignment\aceest-fitness-ci-cd> docker build -t aceest-gym .
[+] Building 10.3s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> transferring dockerfile: 212B
=> [internal] load metadata for docker.io/library/python:3.10-slim
=> [internal] load .dockerignore
=> transferring context: 2B
=> [1/5] FROM docker.io/library/python:3.10-slim@sha256:4ba18b066cee17f2696cf9a2ba564d7d5eb05a91d6a949326780aa7c6912160d
=> resolve docker.io/library/python:3.10-slim@sha256:4ba18b066cee17f2696cf9a2ba564d7d5eb05a91d6a949326780aa7c6912160d
=> [internal] load build context
=> transferring context: 209.20kB
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY requirements.txt
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> [5/5] COPY .
=> exporting to image
=> exporting layers
=> exporting manifest sha256:2e25abef98354064de6ab63a91dac396e4f33ca2d6f7a23fc15f2d763bd4b404
=> exporting config sha256:d2e8ba31e3b37f33d8a46ec723b94050946d8e403b122b4a1668e3516b2668e0
=> exporting attestation manifest sha256:6b38c2d4de20e1a98ea86dd6905ff68dadb3e240f0e1ef073407409f554b1fee6
=> exporting manifest list sha256:6ce68fc0a0a8c7celbb817dce1f12ce2dbccbb11fa9c67765f31134932d2eaalb
=> naming to docker.io/library/aceest-gym:latest
=> unpacking to docker.io/library/aceest-gym:latest
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/ejlbyialskv8bkvdpxuh538ls
(venv) PS D:\M.Tech\BITS Pilani\Semester 3\Introduction to Devops\Assignment\aceest-fitness-ci-cd> docker run -d -p 5000:5000 --name aceest-container aceest-gym
78149c58b8dd458a8afab599695ca54157b771a7d3ab6642d5eabb9ed9b5ca2
```

- App runs on: <http://localhost:5000>
- Container name: aceest-container

Docker Desktop (Running Container):



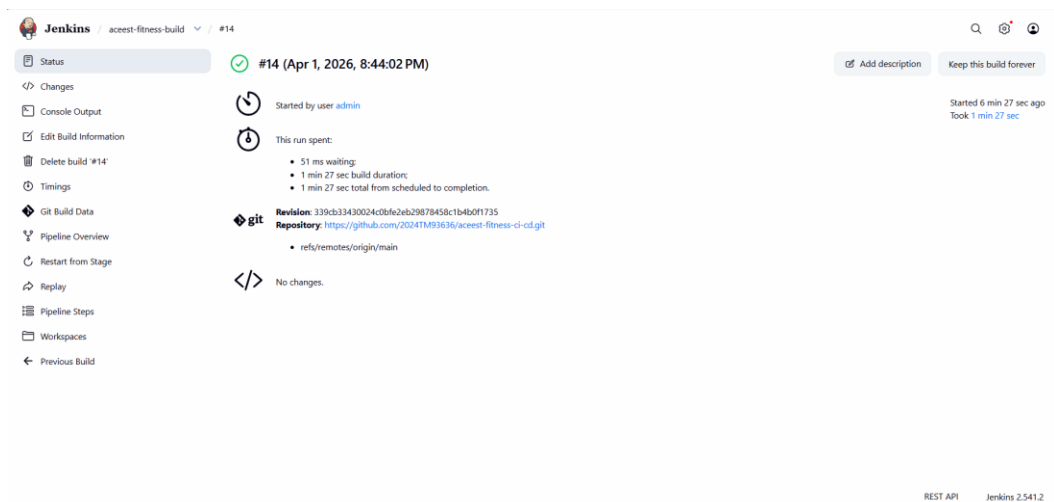
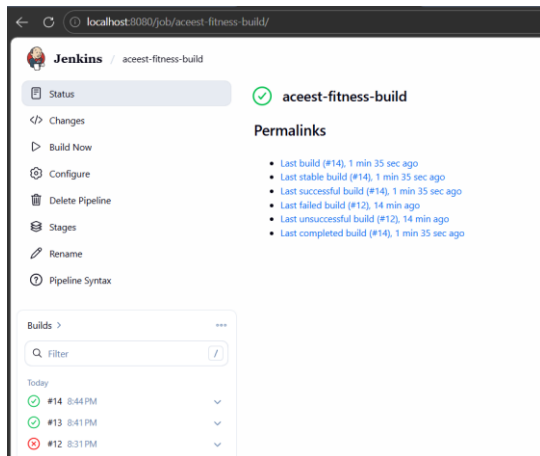
10. Jenkins Pipeline

Jenkinsfile:

- Uses Docker agent python:3.11
- Stages:
 - Check Python version
 - Install dependencies
 - Run Pytest tests
 - Build Docker image
 - Deploy container
- Jenkins acts as a secondary BUILD validation layer.
- It pulls the latest code from GitHub, installs dependencies, runs tests,
- and builds the Docker image in a clean environment to ensure consistency.

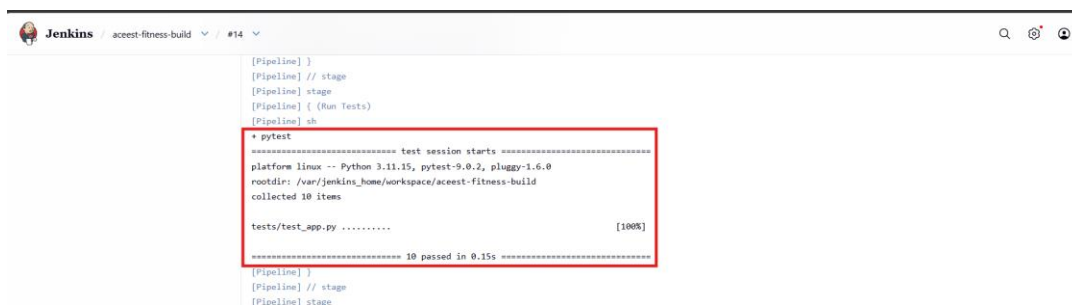
```
(venv) PS D:\M.Tech\BITS Pilani\Semester 3\Introduction to Devops\Assignment\aceest-fitness-ci-cd> docker run -d --name jenkins -p 8080:8080 -p 50000:50000 -v jenkins_home:/var/jenkins_home -v /var/run/docker.sock:/var/run/docker.sock jenkins/jenkins:its f522638fed4aa6ef5d8c2d2498204d53aff9466971e1a25fad09756a29ff349e
(venv) PS D:\M.Tech\BITS Pilani\Semester 3\Introduction to Devops\Assignment\aceest-fitness-ci-cd> docker exec -u root -it jenkins bash
root@f522638fed4a:/# apt update
apt install -y docker.io
Get:1 http://deb.debian.org/debian trixie InRelease [140 kB]
Get:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
Get:3 http://deb.debian.org/debian-security trixie-security InRelease [43.4 kB]
Get:4 http://deb.debian.org/debian trixie/main amd64 Packages [9671 kB]
Get:5 http://deb.debian.org/debian trixie-updates/main amd64 Packages [5812 B]
Get:6 http://deb.debian.org/debian-security trixie-security/main amd64 Packages [119 kB]
Fetched 10.0 MB in 3s (3690 kB/s)
17 packages can be upgraded. Run 'apt list --upgradable' to see them.
Upgrading:
  libcap2
Installing:
  docker.io
Installing dependencies:
  apparmor          gettext-base      libintl-xs-perl   libnftnl11        libsort-naturally-perl  python3-minimal
  containerd        iptables          libip4tc2         libnsl-3-200      libsystemd-shared      python3-protobuf
  criu              libpam-cap        libip6tc2         libnss-systemd    libterm-readkey-perl   python3-pycru
  dbus              libapparmor1      libjansson4       libpam-systemd    libtirpc-common        python3.13
  dbus-bin           libbpf1           libjson-c5        libtirpc3t64     libtirpc3t64          python3.13-minimal
```


Pipeline Screenshot:



Testing Inside Container:

- Tests are executed during CI to validate application logic.
- Docker image build ensures the application runs consistently in a containerized environment.

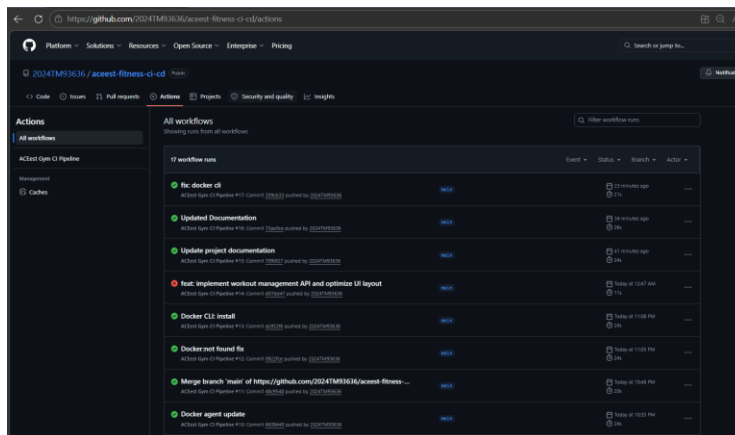


11. GitHub Actions CI Pipeline

Workflow (ci.yml)

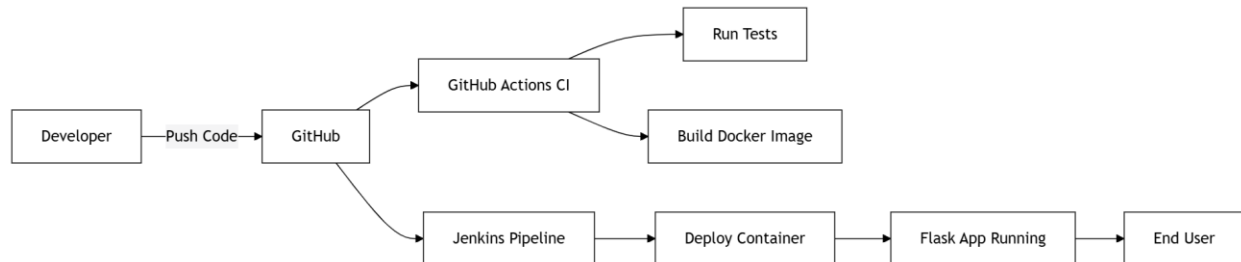
- Triggered on push or pull request to main
- Steps:
 - Checkout repository
 - Setup Python 3.10
 - Install dependencies
 - Run tests
 - Build Docker image
 - Linting and syntax validation are performed to ensure code quality before testing.

GitHub Actions Screenshot:



12. CI/CD Workflow Diagram

This workflow ensures continuous integration through GitHub Actions and continuous delivery through Jenkins, achieving automated and reliable software deployment.



GitHub Actions:

Ensures early detection of issues through automated testing and build validation on every code push.

Jenkins:

Handles continuous deployment by building Docker images and running containers automatically.

13. DevOps Practices Implemented

- Version control with Git & GitHub
- Shift-left testing with Pytest
- Containerization using Docker
- CI/CD with GitHub Actions and Jenkins
- Infrastructure as Code (Dockerfile & Jenkinsfile)

14. Future Enhancements

- Add persistent database integration (PostgreSQL/MySQL)
- Input validation and error handling
- Authentication & authorization
- Logging and monitoring
- Docker image optimization with multi-stage builds
- Kubernetes deployment for scalability

15. Conclusion

This project successfully demonstrates the implementation of a complete DevOps pipeline integrating development, testing, and deployment, ensuring faster and reliable software delivery.

16. References

- GitHub Repository: <https://github.com/2024TM93636/aceest-fitness-ci-cd>
- Tools: Flask, Docker, Pytest, Jenkins, GitHub Actions